

一、四元数相乘 00:21

1. 两个四元数相乘得到一个新的四元数 00:27



唐老师系列教程-四元数

四元数相乘

$q3 = q1 * q2$

两个四元数相乘得到一个新的四元数

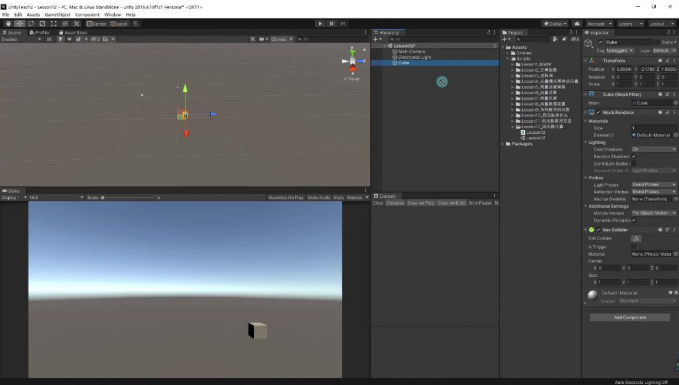
代表两个旋转量的叠加

相当于旋转

注意：旋转相对的坐标系 是物体自身坐标系

-
- 数学表示： $q3 = q1 * q2$ ，表示两个四元数相乘得到一个新的四元数
- 物理意义：代表两个旋转量的叠加，相当于连续进行两次旋转
- 坐标系说明：旋转操作是相对于物体自身的坐标系进行的
- 应用优势：相比欧拉角旋转，可以避免万向节死锁问题
- 初始化方法：实际开发中通常使用轴角对API（Quaternion.AngleAxis）初始化四元数，而非直接使用公式

2. 实例操作 01:09



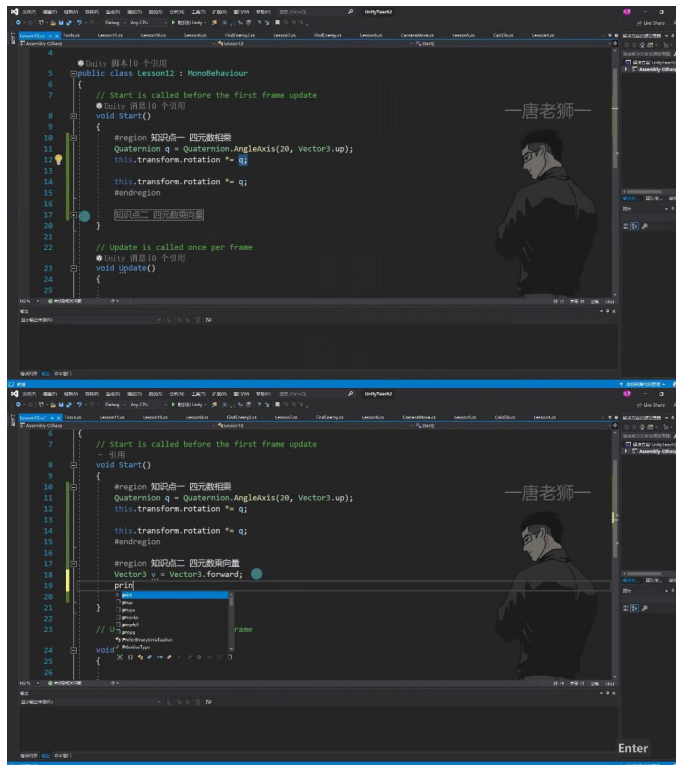
-
- 环境准备：
 - 创建立方体对象
 - 挂载Lesson12脚本控制旋转
- 代码实现：
- 旋转效果：
 - 每次执行使物体绕自身y轴旋转20度
 - 连续执行两次相当于旋转40度
- 坐标系验证：
 - 当物体z轴旋转90度后，y轴朝向左侧
 - 再次执行仍绕物体自身y轴（此时为世界坐标系左侧方向）旋转
 - 证明旋转确实基于物体自身坐标系
- 注意事项：
 - 旋转效果应以四元数计算结果为准，欧拉角显示可能有差异
 - 实际开发中应避免直接修改欧拉角来旋转物体

二、四元数乘向量 04:38

1. 四元数乘向量返回一个新的向量 04:44

- **运算规则：** 四元数乘向量公式为 $v_2 = q_1 * v_1$ ，其中 q_1 是四元数， v_1 是原始向量，运算结果 v_2 是一个新向量
- **功能作用：** 该运算可以将指定向量旋转对应四元数的旋转量，相当于对向量进行旋转操作
- **顺序要求：** 必须四元数在前、向量在后，顺序不可颠倒，否则会报错

2. 实例操作 05:19



- **基础示例：**
 - 初始向量： $Vector3.forward$ 即(0,0,1)
 - 旋转操作：使用 $Quaternion.AngleAxis(45, Vector3.up)$ 创建旋转45度的四元数
- **旋转效果：**
 - 第一次旋转45度后向量变为(0.7,0,0.7)
 - 第二次旋转45度后向量变为(1,0,0)
- **几何解释：**
 - 在xz平面中，初始向量沿z轴正方向
 - 旋转45度后x、z分量相等，符合直角三角形特性
 - 旋转90度后完全指向x轴正方向
- **实际应用：**
 - 可用于实现飞机发射扇形/环形子弹
 - 通过基础方向向量乘以不同旋转角度的四元数，可获得多个发射方向

三、总结



唐老师系列教程-四元数

总结

- 1.四元数相乘——**角度叠加**
- 2.四元数乘向量——**旋转向量**

-
- **核心要点:**
 - 四元数相乘：实现角度叠加效果
 - 四元数乘向量：实现向量旋转功能

四、练习题 10:07

![[练习题要求]](/00_10_10.jpg "练习题要求">

- **题目一:**
 - 模拟飞机发射不同类型子弹的方法
 - 包括单发、双发、扇形、环形四种模式
- **题目二:**
 - 实现摄像机跟随效果
 - 具体要求:
 - 摄像机在人物斜后方，通过角度控制倾斜率
 - 鼠标滚轮控制摄像机距离（有最大最小限制）
 - 摄像机看向人物头顶上方可调节位置
 - 使用`Vector3.Lerp`实现平滑跟随
 - 使用`Quaternion.Slerp`实现朝向过渡

五、知识小结

知识点	核心内容	考试重点/易混淆点	难度系数
四元素相乘	两个四元素相乘得到新四元素，代表旋转量的叠加，基于 物体自身坐标系 旋转。	注意与欧拉角旋转的区别（避免万向节死锁）， 顺序不可逆 （四元素×四元素）。	★★
四元素乘向量	四元素×向量返回新向量，实现向量旋转（如子弹发射方向计算）。	顺序强制要求 （四元素在前，向量在后），旋转基于 向量起点方向 。	★★
四元素初始化	通过 轴角对API （角度+旋转轴）初始化，避免	旋转轴需明确坐标系（如 <code>Vector3.up</code> 实际依赖物体自身坐标系）。	★

	手动公式计算。		
应用场景	游戏中物体旋转（如立方体）、 方向向量生成 （如扇形子弹弹道）。	需结合坐标系理解旋转效果（如物体局部坐标系 vs 世界坐标系）。	★★★